DEPI Digital Egypt Pioneers Initiative



EDUVANCE AI / PROJECT DOCUMENTATION

EDUVANCE AI

System Analysis & Design

Architecture layers, AI pipelines, data model, APIs and implementation state

TRACK	AI & Data Science - Agentic AI and Generative AI System Developer
DOCUMENT	04 - Project Documentation
SOURCE BASIS	Approved Project Plan, existing WBS documents and repository code

01

Problem and architectural objectives

WBS 2.1 and the approved Plan.md

The designed system takes bounded training PDFs, builds a source-traceable course, generates learning assets, supports grounded tutoring, records assessments and learner events, and applies explainable revision rules. A central workflow controller coordinates dependent services through typed contracts.

Bounded source material

Text PDFs; initial 3-file / 50-page limit.

Source traceability

Document, page and content-chunk relationships.

AI orchestration

Controlled agent interactions, state transitions and retries.

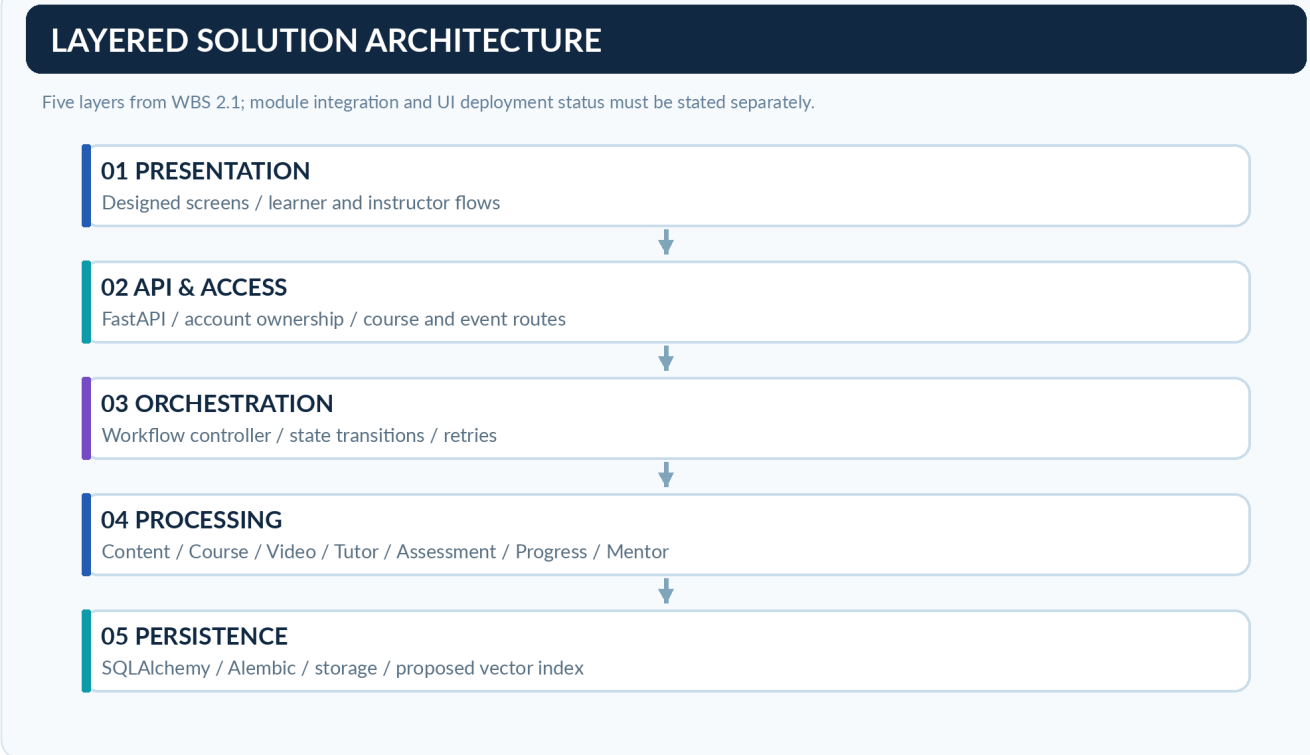
Deterministic decisions

Objective scoring and progress rules separate from probabilistic generation.

02

Five-layer architecture

Layered system topology recorded in WBS 2.1



Architecture overview: target layers and interaction responsibilities; actual integration status is described separately.

Layer	Components in source design	Function
01 / Presentation	Document upload, course review, video player, tutor chat, quiz and dashboard	Learner and instructor interactions; designs exist, frontend code is not yet visible.

Layer	Components in source design	Function
02 / API	FastAPI REST endpoints, authentication, user isolation, event ingestion	Validated request entry, account access and HTTP contracts.
03 / Orchestration	Central Workflow Controller, state / retry tracker	Sequence dependent tasks, control transitions, track failures.
04 / AI processing	Content, Course Designer, Video, Tutor, Assessment, Progress and Project Mentor components	Extraction, generation, retrieval, assessment and recommendations.
05 / Persistence	Relational DB, vector/chunk indexing concept, asset storage	Users, courses, events, structured artifacts and saved files.

1 Learner / instructor
2 FastAPI routes
3 Workflow Controller
4 Agent pipelines
5 DB + file storage

Design vs current implementation

The layered diagram is the project architecture design, not proof of an end-to-end released application. Current main includes the FastAPI backend, database models and several AI pipelines; Frontend and Tutor_RAG remain skeletal.

03 Agent and component responsibilities

Contracts defined in WBS 2.1

Component	Input / responsibility	Defined output
Content Agent	Validate PDFs; extract pages, chunks and concepts	Source-linked text segments and concept relationships
Course Designer	Organize extracted concepts and prerequisite relationships	Modules, lessons, objectives and editable plan
Video Pipeline	Use approved lessons to build scripts, slides, narration and captions	Short generated lesson media
RAG Tutor	Retrieve course-specific evidence and compose grounded answers	Answer with sources or abstention
Assessment Agent	Create objective questions and practical exercise rubrics	MCQ / T-F / exercise specifications
Progress & Recommendation	Consume quiz/activity events and deterministic rules	Progress / concept flags / revision suggestions
Project Mentor	Use studied concepts to recommend a bounded project	Brief, steps and rubric

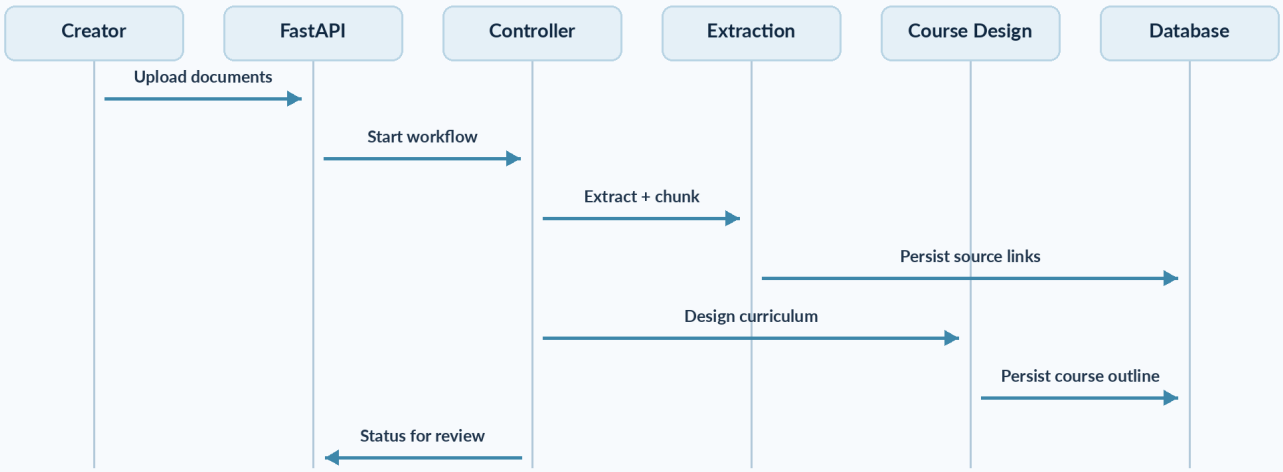
WBS 2.1 defines seven logical agent/service contracts; some are deterministic components rather than LLM agents. The separation reflects the project source documents and should not be misrepresented as seven independent deployed models.

04 Main process and data flow

Based on Plan.md workflow

DOCUMENT-TO-COURSE SEQUENCE / TARGET DESIGN

Illustrative sequence grounded in the approved workflow; not a captured running trace.



Planned logical sequence from user upload to course review; not a verified end-to-end execution log.

1. Upload PDF
2. Extract and chunk
3. Design course
4. Review / approve
5. Generate assets
6. Study + assess

Stage	Incoming information	Transformation and output
Ingestion	User-selected training PDFs	Validation and source file metadata
Extraction	Accepted documents	Page text, chunks, concepts and anchors
Course design	Concepts and prerequisites	Course, modules, lessons and learning objectives
Review gate	Generated course structure	Approved/edited structure before asset generation
Asset generation	Approved lessons	Video, slides, caption data and assessment content
Learning interaction	Lesson access and learner questions	Video interaction and grounded tutor response
Assessment and progress	Quiz attempts and learning events	Deterministic scores, progress and revision suggestions
Project mentor	Completed course concepts	Practical project brief and rubric

Data flow boundary

The last three learning interactions are part of the target complete workflow; individual source modules and backend APIs exist, but the full UI/RAG-integrated learner journey has not been established from the current repository.

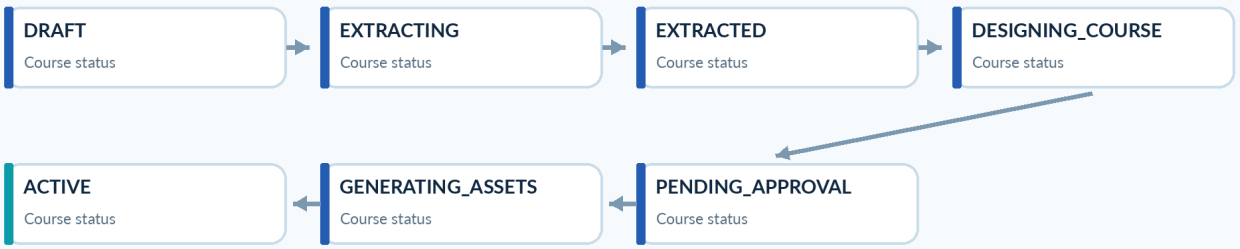
05

Workflow state machine

Target state design compared with current model statuses

COURSE STATE PROGRESSION

State labels from the current ORM/controller; WBS conceptual labels are not identical.



Task statuses: PENDING / RUNNING / COMPLETED / FAILED. Retry budget in controller: MAX_RETRIES = 2.

Implemented status labels as represented in the current ORM, with overall progression simplified for readability.

Designed progression (WBS 2.1): UPLOADED -> EXTRACTING -> EXTRACTED -> DESIGNING_COURSE -> COURSE_PENDING_APPROVAL -> GENERATING_ASSETS -> COURSE_ACTIVE. Additional learning and quiz events follow activation.

Implemented course status field: the current ORM records DRAFT, EXTRACTING, EXTRACTED, DESIGNING_COURSE, PENDING_APPROVAL, GENERATING_ASSETS, ACTIVE, and FAILED. The controller also handles ASSETS_FAILED on retry exhaustion.

Job state	Recorded behavior
PENDING	Job registered but not running
RUNNING	Task execution started
COMPLETED	Task finished
FAILED	Retry budget exhausted / task failed

The controller code defines **MAX_RETRIES = 2**. On a failed attempt, a task may be re-queued until the retry budget is consumed. The original WBS state diagram and the implemented code use slightly different status labels; a formal final diagram should follow the deployed constants when integration is completed.

06

Database design and logical entities

Current ORM snapshot, not an invented schema

The current `Database/models/all_models.py` contains **16 SQLAlchemy model classes** grouped into the following functional areas:

Area	Implemented models	Role
Identity and courses	User, Course, Document	Accounts, ownership, uploaded source files
Source intelligence	ContentChunk, Concept	Extracted content and concepts
Curriculum and media	Module, Lesson, VideoAsset	Course structure and rendered lesson assets

Area	Implemented models	Role
Assessment and mentor	Quiz, QuizQuestion, PracticalExercise, ProjectBrief	Questions, rubrics and proposed projects
Learning data	LearnerEvent, LessonProgress, ConceptMastery	Observed activity, progress and concept scores
Execution	WorkflowTask	Tracked jobs, states and retries

Course ownership

User -> Course -> Documents, Modules and WorkflowTasks

Source lineage

Document -> ContentChunk; Course -> Concept; source-linked assessment metadata

Learning hierarchy

Course -> Module -> Lesson -> Video / Quiz / Exercise

Activity & tracking

Users and course lessons relate to events, lesson progress and concept mastery

Conceptual ERD discrepancy

WBS 2.1 draws QUIZ_ATTEMPTS as a conceptual entity. A `QuizAttempt` model was not found among the 16 classes in the current ORM file; do not describe this table as implemented. The final ERD should be generated from the actual ORM/ migration when approved.

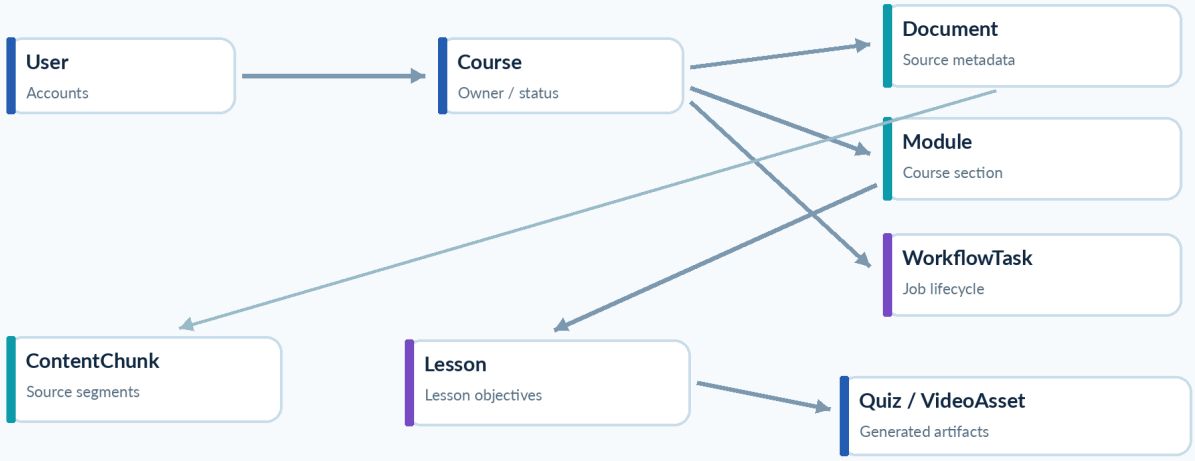
07

Entity relationships - readable ER overview

Source-based relational overview; detailed source remains the model file

RELATIONAL DATA MODEL / KEY RELATIONSHIPS

Readable subset of relationships present in Database/models/all_models.py; not a full generated ERD.



Other ORM entities: Concept, QuizQuestion, PracticalExercise, ProjectBrief, LearnerEvent, LessonProgress, ConceptMastery.

Relationship overview extracted from the current SQLAlchemy model associations.

Parent	Relationship	Child / dependent
User	creates / owns	Course
Course	contains	Document
Document	splits into	ContentChunk
Course	has	Module
Module	groups	Lesson
Lesson	renders	VideoAsset
Lesson	assesses through	Quiz and PracticalExercise
Quiz	contains	QuizQuestion
Course	defines	Concept and WorkflowTask
Course	may have	ProjectBrief
User / Course	record	LearnerEvent
User / Lesson	track	LessonProgress
User / Concept	track	ConceptMastery

Persistence uses SQLAlchemy 2.0 model classes, Alembic migration `0001_initial_schema.py` and `Database/session.py`. The database README reports SQLite validation; PostgreSQL requires separate live validation.

08

API surface and access model

Confirmed from route definitions on current main

Domain	Available route family	Current scope
Health	GET /health/live ; GET /health/ready	Liveness and database schema readiness
Account	POST /api/v1/auth/register ; POST /api/v1/auth/login ; GET /api/v1/auth/me	Registration, login and current-user identity
Courses	POST /api/v1/courses ; GET /api/v1/courses ; GET /api/v1/courses/{course_id}	Create, list and read owned course containers
Documents	GET /api/v1/courses/{id}/documents ; GET .../{document_id}/download	List/download already stored source records
Events	POST /api/v1/events ; GET /api/v1/events/course/{course_id}	Record and list learning events
Progress	GET /api/v1/courses/{id}/progress	Rule-derived course progress
Workflow	GET /api/v1/courses/{id}/workflow ; POST /api/v1/courses/{id}/approve	Inspect state and approve the course

Important endpoint limit

The current document router lists and serves existing document records but does not itself implement the full PDF upload/extraction endpoint. That integration is a designed requirement rather than a confirmed available route.

09

Technology selection and implementation state

Explicitly separating architecture choices from verified code

Technology / tool	Evidence in materials	Status
Python / FastAPI	Backend requirements, app factory and API routers	Code present
Pydantic v2	Schemas and settings used by backend/ modules	Code present
SQLAlchemy / Alembic	ORM classes and migration history	Code present
SQLite	Local configured/default database and test fixtures	Validated locally in documentation
PostgreSQL	Selected target in architectural WBS	Not live-validated in available docs
FAISS / pgvector	Proposed vector indexing alternatives in WBS	No deployed index confirmed
React / Tailwind / shadcn	WBS 2.2 planned UI implementation stack	Designed; frontend code not visible
PDF parsing / slide / TTS tooling	Extraction and Video Pipeline module files	Component code present

10

Security, validation and quality controls

What the source code and plan establish

- Account registration/login and bearer token checks are implemented in `Backend/auth.py`; course ownership checks are used for protected records.
- The current backend uses input validation and safe public error responses; stored document delivery checks course ownership before accessing files.
- PDF ingestion and source traceability are part of the design and Extraction module; do not claim that every integration step is live.
- Generated tutor answers are planned to include sources and appropriate abstention; automated and human evaluation are required.
- Progress and assessment scores use deterministic logic rather than letting an LLM decide mastery.
- The existing tests cover backend, document extraction, course generation, media generation and assessment components; tests were not executed during this documentation review.

11

Current implementation snapshot and remaining integration

Factual inventory based on repository contents

Workstream	Repository evidence	Interpretation
Backend and security	Backend/main.py, auth.py, courses.py, documents.py, events.py, progress.py	Substantial API code present
Database	Database/models/, Database/migrations/, Database/session.py	16 ORM classes and migrations present
Document extraction	AI_Modules/Extraction/ with pipeline and tests	Module implementation present

Workstream	Repository evidence	Interpretation
Course design	AI_Modules/CourseDesign/ with pipeline and tests	Module implementation present
Video generation	AI_Modules/Video_Pipeline/ with pipeline and tests	Module implementation present
Assessment generation	AI_Modules/Assessment/ with pipeline and tests	Module implementation present
Tutor_RAG	AI_Modules/Tutor_RAG/.gitkeep	Directory placeholder, no module code on main
Frontend	Frontend/.gitkeep	Directory placeholder, no UI source on main
Project Mentor	Contract and plan documentation	Independent implementation not established

The complete platform is an integration target. Presence of individual modules and test scripts is evidence of work products, not proof of successful end-to-end execution at the time of this report.

12

Design source register

Traceability to original project material

S1 - Project goals, scope, workflow and acceptance targets
Plan.md - §§1-3, 8

S2 - Layered architecture, state diagram, ERD and seven contracts
members/member_1/Task_WBS_2.1_Architecture_and_Contracts.md

S3 - Personas, user journeys, interface screens, component tokens
members/member_4/Task_WBS_2.2_UI_UX_Wireframes_and_Journeys.md

S4 - Database ORM and Alembic
Database/models/all_models.py; Database/migrations/versions/0001_initial_schema.py

S5 - Current REST API and services
Backend/main.py; Backend/{auth,courses,documents,events,progress}.py

S6 - AI implementation and tests
AI_Modules/{Extraction,CourseDesign,Video_Pipeline,Assessment}/

Document integrity

This document re-expresses design and implementation facts from the supplied sources without changing the project plan or WBS. Any diagrams are explanatory summaries of existing artifacts, not newly approved system features.